Building a Root Cause Analytics Engine with Prometheus, Grafana, and MCP

The pattern I built for a client, rebuilt as a demo you can run

`your name` · Kubernetes Day / CNCF community talk

Alerts tell you users are hurting, not why

An alert says people are affected. It does not say what changed first, which service is the source, how far the damage spread, or what to look at next. You work that out yourself, at 3am, across dashboards, logs, and runbooks.

The on-call loop

```
alert → dashboard → logs
→ another dashboard → runbook
→ human hypothesis
```

Where this came from

This pattern came out of work for a fintech client under NDA. No company name, no topology, no production data, no volumes, no vendors, no real incidents in this talk. Everything runs on synthetic data.

The obvious fix: point an agent at the logs

```
Alert → agent → raw logs + kubectl → incident summary
```

The broad first question	What you get back
“Why is checkout failing?”	which service, which labels, which window?
large raw result set	slow, expensive, rate-limited
partial evidence	a confident story with nothing behind it

> No prompt makes an unbounded evidence surface cheap, predictable, or safe.

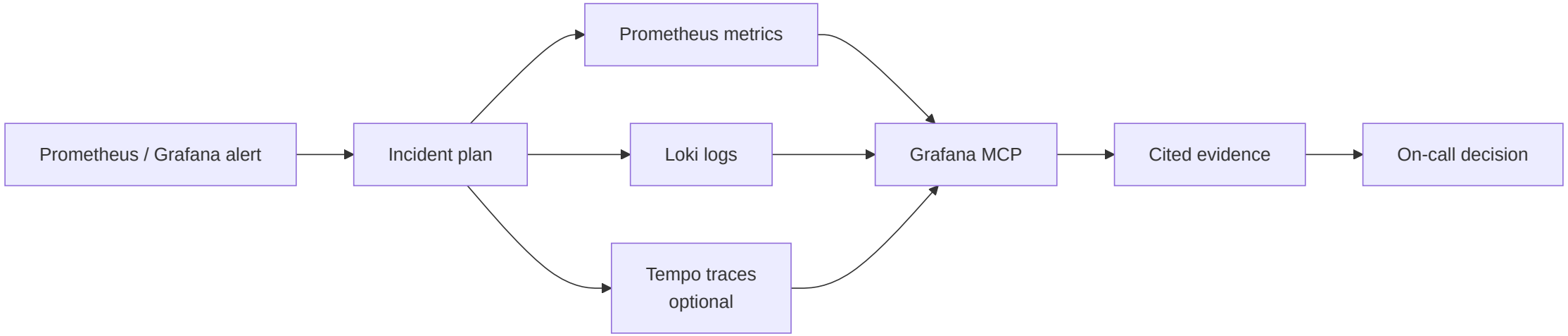
The design pivot: bound the evidence first

```
What bounded evidence answers the first five minutes?
```

Metrics answer the cheap questions first: what failed, when it started, which services moved together, what they probably share. Logs come second, to test a hypothesis that is already narrow.

Telemetry dimension	What it buys you
service, status, latency	scope and severity
dependency direction	a causal hypothesis
deploy or change marker	what changed
trace identifier, where available	request-level proof

Grafana is the evidence plane



Prometheus, Loki, and optional Tempo remain the sources of truth. Grafana is the plane where their evidence lines up. An MCP gateway sits in front of it and exposes a short list of approved, read-only questions.

From client system to something you can clone

At the client	In this repo
bounded Prometheus and Loki evidence queries	fixed templates over a three-service demo
incident scope, state, and evidence contracts	a local persisted workflow with structured findings
human approval before remediation	an explicit operator transition in the CLI
production integrations, identities, policy	contained fault injection, no production credentials

The repo demonstrates the controls and the shape of the workflow. It does not copy client topology, telemetry, volumes, or incident data.

The model never writes PromQL

```
const queries = {
  current_incidents:
    'ALERTS{alertstate="firing"}',
  service_health: (service) =>
    `service:http_error_ratio:rate5m` +
    `{service="${service}"} or ` +
    `service:http_p95_latency_seconds:rate5m` +
    `{service="${service}"}`,
};

function assertService(service) {
  if (!['checkout', 'orders', 'payments']
    .includes(service)) throw new Error('unknown service');
}
```

What the gateway does

It picks from reviewed recording-rule templates and checks the service against an allowlist. The model chooses from a menu, it never authors a query.

The demo hard-codes three services. Production scope comes from the persisted incident plan and the service catalog.

Tool schemas are executable policy

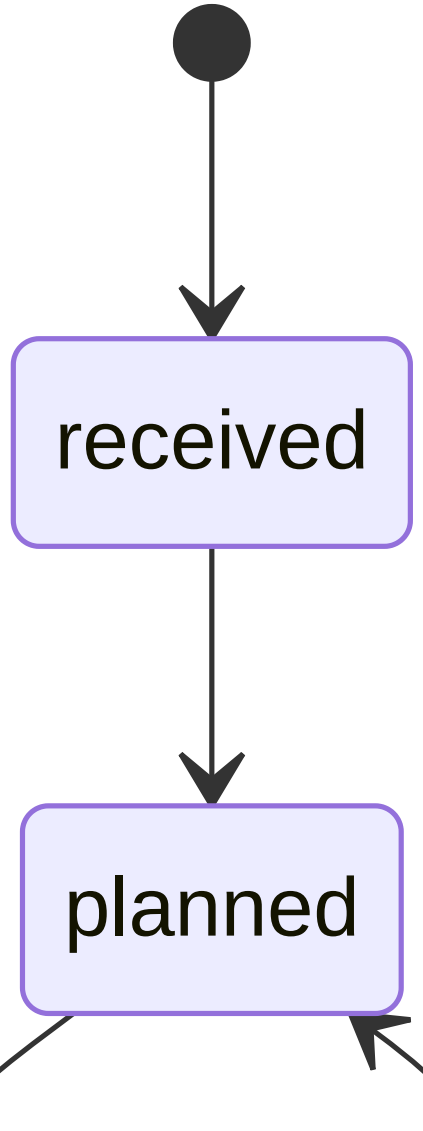
```
{
  name: 'service_health',
  inputSchema: {
    type: 'object',
    properties: { service: {
      type: 'string',
      enum: ['checkout', 'orders', 'payments']
    } },
    required: ['service'],
    additionalProperties: false
  }
}
```

The boundary

- typed, read-only evidence tools
- source, service, time, and result caps on every call
- every finding cites the query that produced it
- no shell, no kubectl, no arbitrary PromQL or LogQL, no dashboard mutation

Illustrative implementation. The public repo includes the bounded Loki tools; production can add Tempo with the same input model.

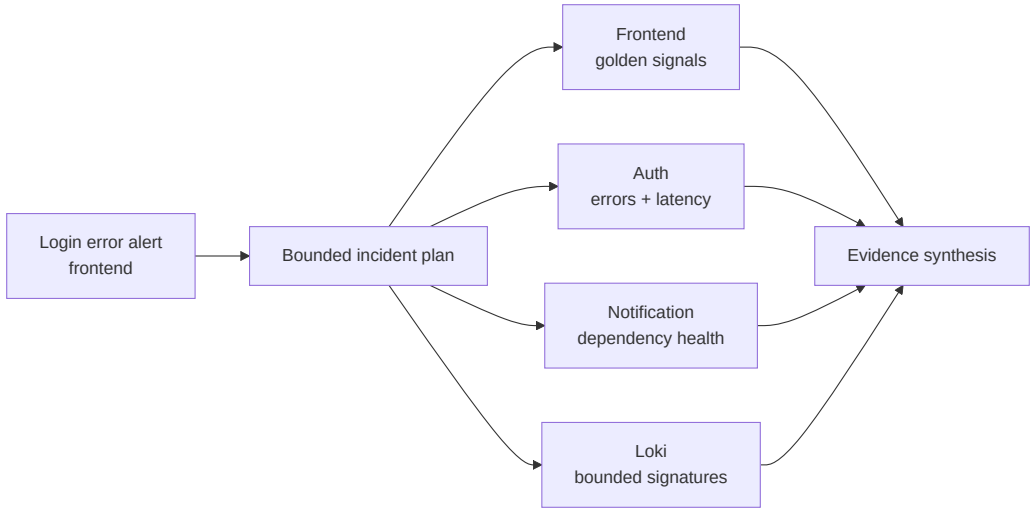
Every state change is a decision someone made



The alert writes a plan before any query runs

```
{
  "incident_id": "inc_01J...",
  "alert": "LoginErrorRateHigh",
  "window": ["08:00Z", "09:00Z"],
  "services": ["frontend", "auth", "notification"],
  "allowed_sources": ["metrics", "logs"],
  "max_parallel_tasks": 6,
  "query_budget": 16
}
```

Reference design, not client code.



A login symptom can justify these checks. It does not justify querying every service in the estate.

Five workers, five narrow questions

Worker	It may ask	It may not
metrics	error ratio, p95, saturation	arbitrary PromQL
logs	bounded Loki error signatures	broad tenant-wide search
traces	exemplars, if Tempo exists	unbounded trace scans
change context	approved annotations near the alert	deploy or roll back
Kubernetes validation	targeted read-only state	`exec`, secrets, write verbs

Every task gets one source domain, a deadline, a result cap, and an explicit scope.

Evidence either supports the hypothesis or rejects it

```
{
  "source": "grafana-mcp",
  "domain": "loki_logs",
  "service": "auth",
  "time_range": ["08:00Z", "09:00Z"],
  "tool": "get_top_error_signatures",
  "query_ref": "restricted://evidence/q_...",
  "confidence": 0.88,
  "limitations": ["No Tempo traces"]
}
```

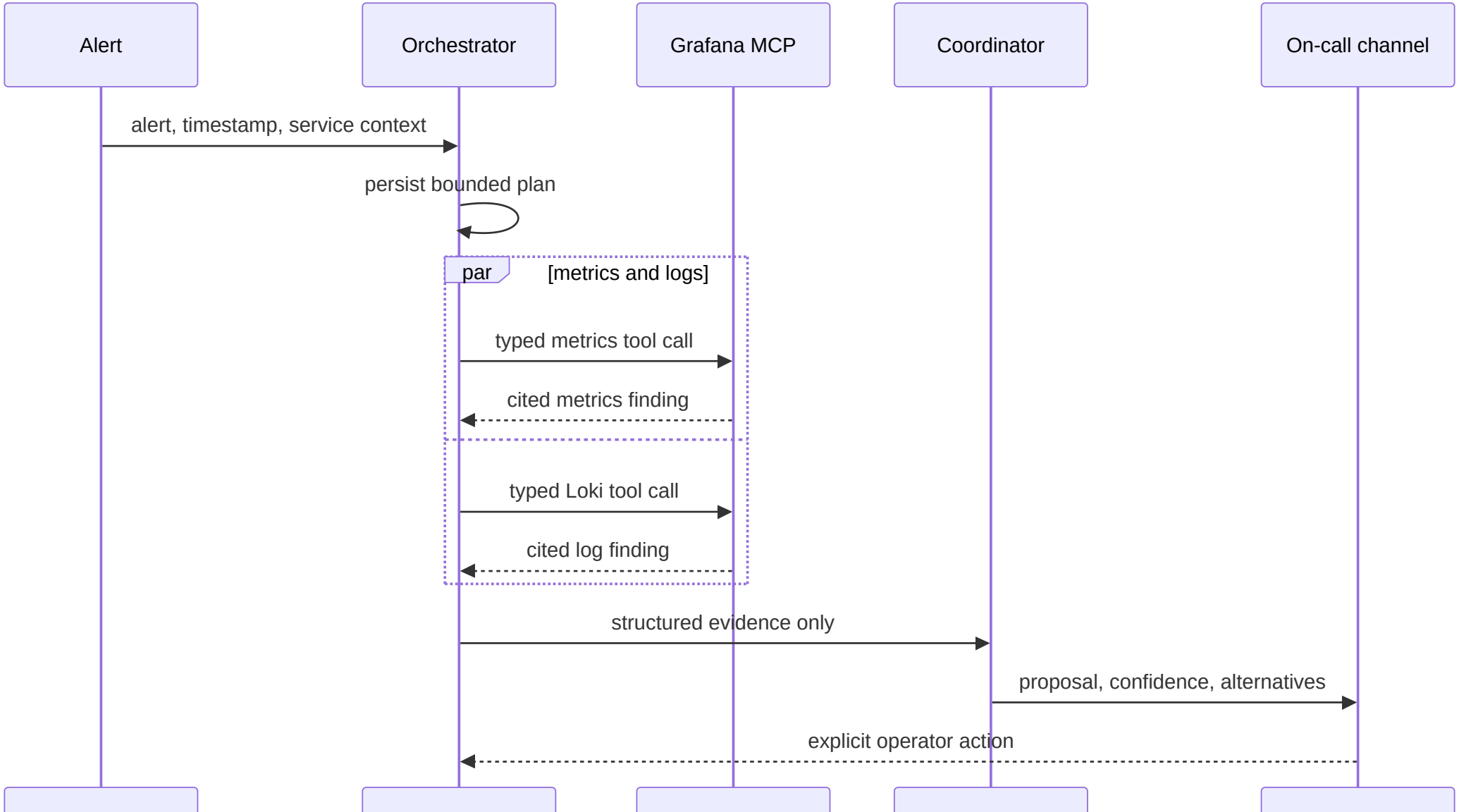
```
Hypothesis: auth JWT validation failure
✓ auth logs precede frontend 5xx
✓ auth error rate rises first
✓ frontend depends on auth

Alternative: notification failure
✗ no onset before alert
? insufficient direct evidence

Proposal: validate auth pod state
```

The coordinator reports citations, alternatives, and what it could not check. You get a bounded hypothesis, not a proven root cause.

The whole investigation, start to finish



Nothing in this design trusts a prompt

Component	Enforced boundary
orchestrator	schedules tasks and persists state, no Grafana source authority
MCP evidence gateway	reads only approved Prometheus/Loki/Tempo datasources
coordinator	reads evidence and writes proposals, no source credentials
Kubernetes validator	separate namespace-scoped read-only identity
every task	bounded window, timeout, concurrency and payload budget
incident record	immutable query, transition, actor, and evidence audit events

No shared admin token, no generic agent credential, no remediation identity anywhere in the investigation path.

Chat buttons request transitions, the server decides

```
function transition(incident, next, actor) {
  if (!transitions[incident.state]?.includes(next))
    throw new Error('invalid transition');
  audit({ incident, next, actor });
  return { ...incident, state: next };
}
```

Reference design, abridged.

```
Likely: auth JWT validation failure
Evidence: metrics q_17, Loki q_22

[ Investigate another hypothesis ]
[ Record manual mitigation ]
[ Acknowledge resolution ]
[ Generate RCA draft ]
```

Slack is one reference implementation. Telegram or any on-call channel can use the same signed, state-checked actions.

A human fixes it, then the system validates

```
evidence-backed proposal
→ on-call engineer chooses action
→ engineer remediates manually
→ system gathers validation evidence
→ team acknowledges outcome
```

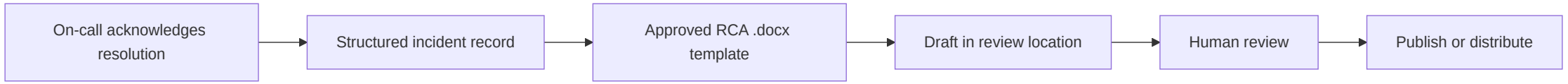
> Automated minor patches are a future path, and they stay disabled.

```
known service + known window + hypothesis
```

```
kubectl -n auth get pods -l app=auth
kubectl -n auth get events --sort-by=.lastTimestamp
```

No `exec`, no secrets, no writes, no broad first-hop log scraping.

The RCA is a draft until a person reads it



The draft assembles itself from the alert, the evidence, the hypotheses, the actions, the validation, and the acknowledged outcome. A person reads it before anyone else sees it.

Why this beats the raw-log agent

Raw-log agent	Evidence-first RCA
unbounded first question	a persisted incident plan
broad raw-log search	Prometheus and Grafana as the first hop
arbitrary retrieval surface	typed read-only MCP tools
the story can outrun the evidence	cited findings and alternatives
hidden retry behavior	explicit state transitions
the agent is the workflow	the on-call team decides

What it cannot do

- Metrics give you correlation. Causation is still your job.
- It cannot fix missing instrumentation, unclear ownership, or a dependency map nobody maintains.
- Request-level questions still need logs and traces.
- It does not authorize remediation and it does not replace an experienced responder.

If you build this, build it in this order

Build first	Then add
stable service telemetry and recording rules	bounded Tempo tools and change context
fixed read-only Prometheus and Loki MCP tools	dependency-aware fan-out
a persisted plan and evidence contract	dedicated identities and a durable audit record
explicit on-call approval state	reviewed RCA document generation

> Start with the smallest evidence interface that answers the questions your team asks in every incident.

Takeaways

1. Put Prometheus, Loki, and Grafana at the evidence plane.
2. Use MCP to expose a short list of read-only evidence tools. The model never writes queries.
3. Persist the plan, the evidence, the identities, and every transition, so the humans stay in control.

Run it yourself

Everything you saw runs locally: Prometheus recording rules, a Grafana correlation dashboard, read-only MCP tools, and a contained fault you can flip on and off.

```
repository URL: add when published
```

It is a learning aid derived from the client pattern, with no client identity, topology, telemetry, or incident data.